Core Engine

Squares

Ownership & Econom y

Com m and Layer

P resen tatio n

Random ness

IGameContext

+void sendPlayerToTims(Player &player)
+void promptPurchase(Player &player, Property &property)
+void handleDebt(Player &debtor, int amount, Player *creditor)
+void handleTuition(Player &player)
+DiceResult rollForGym()
+int activeCupCount() const
+void awardCup(Player &player)
+void resolveLanding(Player &player, int squareIndex)
+void logEvent(const std::string &msg)
+SLCEvent generateSLCEvent()
+int generateNeedlesHallDelta()
+bool checkRollUpTheRim()
+TurnPhase getCurrentPhase() const
+Player &currentPlayer()
+std::vector<Player *> getActivePlayers()
+Player *findPlayer(const std::string &name)

WatopolyGame

+void setupNewGame()
+void loadGame(const std::string &filename)
+void run()
+Board &getBoard()
+const Board &getBoard() const
+TurnContext &turnContext()
+Dice &getDice()
+bool isTestingMode() const
+bool isReplayEnabled() const
+EventLogger &eventLogger()
+const std::vector<std::unique_ptr<Player>> &playerList() const
+GameSnapshot snapshot() const
+void redraw ()
+void runAuction(Property &property)
+void saveGame(const std::string &filename) const
+void replayEventsFromFile(const std::string &filename) const
+void printAssets(const Player &player) const
-void setupCommands()
-void processTurn()
-void handleTimsLine(Player &player)
- bool isGameOver() const
-void announceWinner() const

WatopolyApp

+int run(int argc, char **argv)

Board

+Square &getSquare(int pos)
+const Square &getSquare(int pos) const
+int positionOf(const std::string &name) const
+int numSquares() const
+std::vector<MonopolyBlock> &monopolyBlocks()
+const std::vector<MonopolyBlock> &monopolyBlocks() const
+void setSquare(int pos, std::unique_ptr<Square> sq)
+void addBlock(MonopolyBlock block)
+Property *findProperty(const std::string &name)

Player

+const std::string &getName() const
+char getToken() const
+PieceType getTokenType() const
+int getCash() const
+int getPosition() const
+void addCash(int amount)
+bool deductCash(int amount)
+void m oveBy(int steps)
+void m oveTo(int position)
+void addProperty(Property *prop)
+void removeProperty(Property *prop)
+const std::vector<Property *> &properties() const
+int getTotalW orth() const
+bool isInTimsLine() const
+void enterTimsLine()
+void leaveTimsLine()
+int tim sTurns() const
+void incrementTimsTurns()
+int timsCups() const
+void addTimsCup()
+bool useTimsCup()
+void declareBankruptcy()
+bool isBankrupt() const
+DiceResult getLastRoll() const
+void setLastRoll(DiceResult r)
+void setPosition(int pos)
+void setCash(int c)
+void setTimsCups(int n)
+void setTimsTurns(int n)
+void setInTimsLine(bool v)

Dice

+DiceResult roll()
+DiceResult roll(int die1, int die2)
+void resetConsecutiveDoubles()
+int consecutiveDoubles() const

DiceResult

+ in t d ie 1
+ in t d ie 2
+ in t s u m
+bool isDoubles

TurnContext

+int currentPlayerIndex
+TurnPhase phase
+int consecutiveDoubles
+DiceResult lastRoll
+Player *debtCreditor
+int debtAm ount

Square

+std::string nam e() const
+int index() const
+void landOn(Player &player, IGameContext &game)
+bool isProperty() const
+bool isAcademic() const

Property

+Player *getOwner() const
+void setOwner(Player *p)
+int purchaseCost() const
+bool isMortgaged() const
+void m ortgage()
+void unm ortgage()
+bool wasTransferredMortgaged() const
+void setTransferredMortgaged(bool v)
+void landOn(Player &player, IGameContext &game)
+int calculateFee(Player &visitor, IGameContext &game)

AcademicBuilding

+MonopolyBlock *getBlock() const
+int im provem entCost() const
+int im provem ents() const
+void setIm provem ents(int n)
+bool canBuyImprovement() const
+bool buyIm provem ent()
+bool sellIm provem ent()
+int calculateFee(Player &visitor, IGameContext &game)

Residence

+int calculateFee(Player &visitor, IGameContext &game)

G ym

+int calculateFee(Player &visitor, IGameContext &game)

CollectOSAP

+void landOn(Player &player, IGameContext &game)

DCTimsLine

+void landOn(Player &player, IGameContext &game)

GoToTims

+void landOn(Player &player, IGameContext &game)

GooseNesting

+void landOn(Player &player, IGameContext &game)

TuitionSquare

+void landOn(Player &player, IGameContext &game)

CoopFee

+void landOn(Player &player, IGameContext &game)

SLCSquare

+void landOn(Player &player, IGameContext &game)

NeedlesHallSquare

+void landOn(Player &player, IGameContext &game)

MonopolyBlock

+std::string nam e() const
+void addBuilding(AcademicBuilding *b)
+std::vector<Academ icBuilding *> buildings() const
+bool isOwnedBy(Player const &player) const
+bool hasAnyImprovements() const

CommandInterpreter

+void registerCommand(std::unique_ptr<Command> cmd)
+bool executeLine(const std::string &line, W atopolyGame &game)

Command

+std::string cmdName() const
+bool execute(W atopolyGame &game, const std::vector<std::string> &args)
+bool isValidInPhase(TurnPhase phase) const

AllCommand SaveCommand HistoryCommand ReplayCommand RollCommand NextCommand TradeCommand ImproveCommand MortgageCommand UnmortgageCommand BankruptCommand AssetsCommand

DisplayObserver

+void update(const GameSnapshot &snap)

DisplayHub

+void attach(DisplayObserver *obs)
+void detach(DisplayObserver *obs)
+void publish(const GameSnapshot &snap)

BoardDisplay

+void update(const GameSnapshot &snap)

EventLogger

+void setEnabled(bool on)
+bool isEnabled() const
+void log(const std::string &m sg)
+void printHistory(int n = -1) const
+const std::vector<std::string> &events() const

GameSnapshot

+SquareInfo squares[40]
+std::vector<PlayerInfo> players

SquareInfo

+std::string nam e
+bool isProperty
+bool isAcademic
+std::string ownerNam e
+int im provem ents
+bool m ortgaged

PlayerInfo

+std::string nam e
+char token
+int cash
+int position
+int tim sCups
+bool inTimsLine
+bool bankrupt

RandomEventGenerator

+void seed(unsigned value)
+int rollD ie()
+SLCEvent generateSLCEvent()
+int generateNeedlesHallDelta()
+bool rollUpTheRimCheck(int activeCupCount)

SLCEvent

Back3
Back2
B ack1
Forward1
Forward2
Forward3
GoToTims
AdvanceToOSAP

TurnPhase

PreRoll
Rolling
PostRoll
DebtResolution
TuitionDecision

PieceType

Goose
GRTBus

Doughnut
Professor
Student
Money
Laptop
PinkTie

Dependency Inversion: all Squares
depend on this narrow interface,
not on the concrete WatopolyGame.
WatopolyGame implements IGameContext.
Commands depend on WatopolyGame
directly (see design doc for rationale).

Centralises all per-turn mutable state:
- consecutiveDoubles (for triple-doubles jail)
- lastRoll exists in struct but is not used in current flow
(Player::lastRoll is used by Gym / Tims flow)
- debtCreditor (null means owed to Bank)
- debtAmount (debt being resolved)
- currentPlayerIndex and TurnPhase

Board is a pure container.
All 40 squares and 8 MonopolyBlocks are
constructed by the free function
buildDefaultBoard(Board &) — Board itself
has no knowledge of specific square types.
Observer role moved to DisplayHub.

Mortgage state:
- Mortgaged properties charge no fee
- Unmortgage cost = 60% of purchase cost
- transferredMortgaged flag tracks properties
received via trade or bankruptcy transfer
(incur immediate 10% Bank fee; extra 10%
applies if recipient unmortgages later)

DisplayHub replaced Board as the
Observer Subject. It publishes
GameSnapshot structs rather than
passing Board & Player references.

toChar / fromChar are explicit mappings:
Goose=G, GRTBus=B, Doughnut=D, Professor=P,
Student=S, Money=$, Laptop=L, PinkTie=T.

creates

1

1

1

2..6

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

4 0

1

8

1

*

owns

1

*

owner

0..1

1

1

11

2..3

1

*